

CMPE 492
Making RAG Systems Domain-Specific by
Fine-Tuning
Embedders on Synthetic Queries

Hikmet Can Köseoğlu

Advisor:
Bahri Atay Özgövde

Project repository:
<https://github.com/hckoseoglu/cmpe492-rag>

Contents

1	INTRODUCTION	1
1.1	Broad Impact	1
1.2	Ethical Considerations	2
2	PROJECT DEFINITION AND PLANNING	3
2.1	Project Definition	3
2.2	Project Planning	4
2.2.1	Project Time and Resource Estimation	4
2.2.2	Success Criteria	5
2.2.3	Risk Analysis	6
3	RELATED WORK	7
3.1	Retrieval-Augmented Generation	7
3.2	RAG Evaluation: RAGAS and RAGChecker	7
3.3	LLM-as-Judge	8
3.4	Embedding Benchmarks: MTEB and MMTEB	8
3.5	Dense Retriever Fine-Tuning and Hard-Negative Mining	8
3.6	Concept Normalization for Query Enhancement	9
3.7	Vector Quantization: PolarQuant and QJL	9
4	METHODOLOGY	10
4.1	RAG Pipeline Overview	10
4.2	Chunking Strategies	11
4.2.1	Base Chunking (Recursive Character Splitting)	11
4.2.2	Semantic Chunking	12
4.2.3	Parent-Child (Hierarchical) Chunking	12
4.3	Concept Normalisation	13
4.3.1	Vocabulary Extraction	13
4.3.2	Three-Stage Normalisation Pipeline	13
4.4	Cross-Encoder Reranking	14
4.5	Vector Quantisation: TurboQuant	15

4.5.1	Stage 1: Random Rotation and Lloyd-Max Quantisation	15
4.5.2	Stage 2: QJL Residual Correction	16
4.6	Knowledge Probing	16
4.6.1	Lexical Obfuscation	16
4.6.2	Probing Experiment	17
4.7	Synthetic Data Generation	18
4.8	Domain-Specific Triplet Generation	18
4.8.1	Stage 1: Agentic Chunking	18
4.8.2	Stage 2: Positive Pair Generation	19
4.8.3	Stage 3: Hybrid Search and Hard-Negative Mining . .	19
4.8.4	Stage 4: Expert Judge Validation	19
4.9	Retriever Fine-Tuning	20
4.9.1	Hard-Negative Filtering	20
4.9.2	Train/Test Split	21
4.9.3	Contrastive Training	21
4.9.4	Evaluation Protocol	21
4.10	Test Dataset	22
4.11	Evaluation Framework	22
4.11.1	LLM-as-Judge Evaluators	22
4.11.2	Custom Retrieval Benchmark	23
4.11.3	MTEB Fitness Benchmark	24
5	REQUIREMENTS SPECIFICATION	26
5.1	Use Cases	26
5.2	Functional Requirements	27
5.3	Non-Functional Requirements	29
6	DESIGN	31
6.1	Information Structure	31
6.2	Information Flow	32
6.2.1	Chat Request Flow	32
6.2.2	RAG Evaluation Pipeline Flow	32
6.3	System Design	33
6.4	User Interface Design	36
7	IMPLEMENTATION AND TESTING	37
7.1	Implementation	37
7.2	Testing	38
7.2.1	Test Dataset	38
7.2.2	Evaluation Experiments	38
7.2.3	Observability	39

7.3	Deployment	39
8	RESULTS	40
8.1	Chunking Strategy Comparison	40
8.2	Reranker Evaluation	40
8.3	Concept Normalization	41
8.4	Retriever Fine-Tuning	43
8.5	Knowledge Probing via Lexical Obfuscation	47
8.6	Vector Quantization (TurboQuant)	48
9	CONCLUSION	51

Chapter 1

INTRODUCTION

1.1 Broad Impact

Access to qualified personal training has historically been constrained by cost, geography, and availability. Certified fitness professionals command hourly rates that place sustained, individualized guidance out of reach for the majority of the population, while published exercise science literature remains largely confined to academic and professional audiences. This project addresses that gap by developing a Retrieval Augmented Generation (RAG) system that grounds an AI assistant’s responses in authoritative fitness textbooks, including publications by the National Strength and Conditioning Association (NSCA) and the American College of Sports Medicine (ACSM). By making evidence-based exercise programming accessible through a conversational interface, the system lowers the barrier to expert-quality fitness guidance and has the potential to benefit a broad population of individuals who lack access to professional coaching.

Beyond its application to personal fitness, the project contributes to a broader discourse on the responsible deployment of large language models (LLMs) in knowledge-intensive domains. A central challenge in applying LLMs to specialized fields is the tendency of these models to generate plausible but unsupported claims drawn from parametric, training-time knowledge. The RAG architecture adopted in this project directly addresses this limitation by retrieving relevant passages from a curated document corpus at inference time and conditioning the model’s response on those passages. This mechanism renders the system’s outputs traceable to specific source documents, enabling users and evaluators to verify the provenance of any recommendation. In this sense, the project serves as a case study for how retrieval-grounded generation can improve the trustworthiness and auditability of AI

assistants in domains where accuracy carries real-world consequences.

1.2 Ethical Considerations

The domain of physical fitness presents a distinct category of safety risk that must be taken seriously in any AI-assisted application. Erroneous exercise recommendations—such as prescribing inappropriate loads, contraindicated movements, or unsuitable training volumes—can result in acute injury or chronic musculoskeletal harm, particularly for users with pre-existing conditions or limited training experience. For this reason, the system is designed to derive its recommendations from retrieved passages in expert-authored textbooks rather than from the model’s unconstrained parametric knowledge. It bears emphasis that the system is intended as a research tool and educational aid; it does not constitute professional fitness coaching and is not a substitute for assessment or guidance by a qualified health or exercise professional.

The system’s retrieval corpus is composed of copyrighted educational and professional publications. Their inclusion in this research is consistent with fair use principles applicable to academic and non-commercial research, as the texts are used solely for the purpose of building and evaluating a retrieval index and are not redistributed or reproduced in any form that would substitute for the original works. All source materials are properly attributed, and the system’s design does not circumvent access controls or distribution restrictions. These considerations will be revisited as the project matures: any deployment beyond the current research context would require a careful reassessment of licensing obligations and, depending on the jurisdiction and intended user population, may also require review under applicable health and consumer protection regulations.

Chapter 2

PROJECT DEFINITION AND PLANNING

2.1 Project Definition

The central goal of this project is to build a reliable and trustworthy AI Personal Trainer whose responses are grounded in authoritative, non-parametric knowledge drawn from expert-authored exercise science literature. Large language models (LLMs) encode vast amounts of information during pre-training, but this parametric knowledge is static, unattributable, and prone to hallucination—particularly in specialised domains such as physical fitness where incorrect guidance regarding load, exercise selection, or programming volume can cause injury. The RAG architecture directly addresses this limitation: rather than relying on what the model has memorised, the system retrieves relevant passages from a curated corpus of trusted textbooks at inference time and conditions its response on those passages. The result is a system whose outputs are traceable to specific source documents, making claims verifiable and reducing the risk that the model substitutes its own associations for evidence-based guidance.

Beyond building the application, the project pursues a systematic research objective: to identify which combination of RAG pipeline components yields the best retrieval quality and answer trustworthiness in the fitness domain. Industry best practices are surveyed and multiple approaches are tested against each other, including different document chunking strategies, the use of rerankers as a post-retrieval filtering step, fine-tuning of embedding models on domain-specific data, agentic RAG architectures, query enhancement techniques such as concept normalisation, and the use of different vector database backends. In addition, the project develops and validates a

methodology for detecting *parametric knowledge leakage*: conditions under which the model disregards retrieved context and falls back on memorised associations.

The retrieval corpus consists of more than ten fitness-domain PDFs including publications from the NSCA, ACSM, NASM, and additional references covering anatomy, periodization, and hypertrophy. The project does not constitute clinical or rehabilitation advice and is not intended as a substitute for professional coaching.

2.2 Project Planning

2.2.1 Project Time and Resource Estimation

Compute cost. All LLM and embedding calls are routed through the OpenAI API. The primary models are a cost-efficient instruction-tuned LLM for evaluation judging (\$0.15 per million input tokens / \$0.60 per million output tokens) and a high-quality text embedding model (\$0.13 per million tokens). Evaluation LLM calls are the dominant cost driver; parallel execution of evaluator calls substantially reduces wall-clock time compared to sequential invocation. Larger datasets and additional pipeline variants will require proportionally more compute time, making parallelisation essential for experimentation.

Memory. Peak RAM consumption reached 3,748 MB (22.9% of system memory) during a chunking experiment where multiple vector stores were held in memory simultaneously. RAM usage is monitored throughout experiments. Experiments involving larger corpora will require additional memory capacity.

Dataset construction time. The custom benchmark dataset (100 queries, 400 documents) was generated via LLM-assisted HyDE document synthesis and hard-negative distractor generation. At a rate of approximately 50–100 documents per minute with parallelised API calls, the full corpus generation required roughly 10–15 minutes. The 36-question annotated evaluation dataset was curated manually from source textbooks.

The time complexity for increasing the corpus size is $O(n \cdot m)$, where n is the number of documents and m is the number of queries. This linear relationship exists because the relevance of each document-query pair must be determined *a priori* to ensure an accurate retrieval assessment.

2.2.2 Success Criteria

The success of the RAG pipeline is measured through three complementary evaluation methodologies.

LLM-as-judge evaluators. Four evaluators assess end-to-end RAG quality using an LLM as the judge, each returning a boolean verdict and a natural-language explanation:

- (i) **Correctness** — the generated answer is factually consistent with the ground-truth reference answer, with no conflicting statements.
- (ii) **Relevance** — the answer concisely addresses the user’s question.
- (iii) **Groundedness** — the answer is supported exclusively by the retrieved document passages; no claims are introduced beyond what the retrieved context contains.
- (iv) **Retrieval Relevance** — the retrieved chunks contain the specific information necessary to answer the question.

Groundedness directly measures the system’s core objective: that responses derive from trusted, non-parametric knowledge rather than from the model’s memorised associations.

IR retrieval metrics. Retrieval ranking quality is evaluated independently of generation using standard information retrieval metrics: nDCG, Precision, Recall, and Mean Reciprocal Rank (MRR), computed against a curated set of relevance judgements.

Custom domain benchmark. A domain-specific retrieval benchmark is constructed following the MTEB framework, consisting of 100 topically orthogonal fitness queries and a 400-document corpus composed of gold documents generated via HyDE and hard-negative distractors. The primary metric is nDCG@10, enabling direct comparison against standard MTEB retrieval baselines.

Real-world evaluation dataset. A curated dataset of 36 questions, sourced from authoritative fitness textbooks including *Bodybuilding Anatomy*, *Exercise Technique Manual*, *Essentials of Strength Training and Conditioning*, *Science and Development of Muscle Hypertrophy*, *Periodization: Theory and Methodology of Training*, *Anatomy of Movement*, and publications from ACSM and NSCA, provides a direct measure of the AI Personal Trainer’s performance on questions that users may realistically ask. Each question is annotated with a reference answer and the corresponding source passage, enabling direct assessment of end-to-end answer quality.

2.2.3 Risk Analysis

A primary concern is that the LLM may rely on its own parametric knowledge rather than the retrieved context, generating responses that are not grounded in the trusted source documents. This is directly addressed by the groundedness evaluator, which explicitly checks whether every claim in the generated answer can be traced back to the retrieved passages. Responses that introduce information beyond the retrieved context are flagged as ungrounded, providing a concrete signal for this failure mode.

A second risk is that the knowledge hub may not contain a document that adequately addresses the user’s question. To mitigate this, the knowledge hub is assembled to be as comprehensive as possible, drawing from multiple authoritative institutions and covering a wide variety of fitness topics, including strength training, hypertrophy, periodization, anatomy, and exercise technique.

A third risk concerns the reliability of LLM-based evaluation itself: the judge may fail to correctly assess answer quality, either by accepting a hallucinated response as grounded or by incorrectly penalising a valid answer. State-of-the-art evaluation approaches are investigated throughout the project to identify and mitigate the known failure modes of LLM-as-judge methods.

Chapter 3

RELATED WORK

3.1 Retrieval-Augmented Generation

Lewis et al. [8] introduce RAG as a paradigm that combines parametric knowledge (encoded in model weights) with non-parametric knowledge retrieved from an external document index at inference time. A dense retriever fetches the top- k relevant passages, and a seq2seq generator built on the Transformer architecture [12] conditions its output on those passages. This grounds responses in verifiable sources rather than static, potentially incorrect memorised associations—the core motivation for using RAG in the fitness domain, where the model’s parametric knowledge may be incomplete or misleading.

3.2 RAG Evaluation: RAGAS and RAGChecker

Evaluating a RAG system requires assessing both the retriever and the generator independently. RAGAS [4] provides reference-free metrics for this purpose: *answer relevancy*, *faithfulness*, and *context relevance*, each targeting a different failure mode without requiring ground-truth annotations. This project uses RAGAS in `rag_eval/evals.py` for lightweight evaluation runs. RAGChecker [11] offers finer-grained diagnostics via claim-level entailment checking, producing separate scores for retriever and generator quality and correlating more strongly with human judgements. Together, these frameworks enable targeted debugging: poor retrieval and poor generation require different remedies.

3.3 LLM-as-Judge

Both frameworks rely on the LLM-as-judge paradigm, where a language model evaluates another model’s outputs. Han et al. [6] benchmark 54 LLMs as judges and show that Pearson correlation alone is insufficient—Cohen’s Kappa analysis is needed to detect systematic biases such as consistent leniency or harshness. This informs the evaluation design in `rag_eval_langchain/evaluators.py`, which uses `gpt-4o-mini` with structured TypedDict output schemas across four evaluators (*correctness*, *relevance*, *groundedness*, *retrieval relevance*) to enforce consistent, auditable verdicts.

3.4 Embedding Benchmarks: MTEB and MMTEB

MTEB [9] is a comprehensive benchmark covering 8 embedding tasks across 58 datasets, using nDCG with graded relevance scores (0/1/2) as the primary retrieval metric. The project’s `ndcg-eval/` module adopts the same scoring scheme, and the `text-embedding-3-large` model used here has an established MTEB leaderboard position. MMTEB [3] extends this to over 500 tasks across 250+ languages and introduces hard-negative sampling and inter-task correlation-based downsampling, methodology that informs the construction of the project’s custom retrieval benchmark.

3.5 Dense Retriever Fine-Tuning and Hard-Negative Mining

A general-purpose embedding model can be adapted to a specialised domain by continued contrastive training on in-domain query–passage pairs. Reimers and Gurevych [10] introduce the Sentence-BERT bi-encoder architecture and the contrastive training regime—now distributed through the `sentence-transformers` library—that underpins this approach. The dominant objective is *Multiple Negatives Ranking Loss* (MNRL), an in-batch contrastive loss that, for each anchor query, treats its paired positive passage as the only correct answer and every other passage in the batch as a negative; the model is trained to maximise the similarity of the true pair relative to all in-batch alternatives. The quality of this signal depends critically on the negatives: *hard negatives*—passages that are topically and lexically close to the query yet do not answer it—force the encoder to learn fine-grained distinctions that random negatives do not. BGE-M3 [1], the embedding model fine-tuned in this work, is a multilingual, multi-functionality encoder sup-

porting an 8,192-token context window and both dense and sparse retrieval, trained with exactly this hard-negative contrastive recipe.

Mining high-quality hard negatives is itself a retrieval problem. Reciprocal Rank Fusion (RRF) [2] combines the ranked outputs of several retrievers by summing the reciprocals of each document’s rank across lists; because it depends only on ranks and not on raw scores, it robustly fuses retrievers whose scores live on incomparable scales—such as an unbounded BM25 lexical score and a bounded cosine similarity. This project uses RRF to fuse sparse and dense candidate lists when mining hard-negative distractors for the fine-tuning corpus (Section 4.8).

3.6 Concept Normalization for Query Enhancement

Vocabulary mismatch between informal user queries and technical document terminology reduces retrieval accuracy. Kang et al. [7] demonstrate in a biomedical setting that prepending a rule-based NLP normalization module to a dictionary-based concept recognizer improves F-scores by over 10 percentage points. The same principle applies here: users say “abs” while documents say “rectus abdominis”. This motivates the concept normalization in `simple-rag/exercise_filter.py` and the broader exploration in the `concept_normalization/` directory.

3.7 Vector Quantization: PolarQuant and QJL

Storing high-dimensional embedding vectors at scale is memory-intensive. Peak RAM usage in this project reached 3,748 MB during indexing with `text-embedding-3-large` (3072-dimensional vectors). PolarQuant [5] addresses this by transforming KV embeddings into polar coordinates after random preconditioning, achieving $4.2\times$ compression without storing per-block quantization constants. QJL [13] applies a Johnson-Lindenstrauss random projection followed by sign-bit quantization, achieving $5\times$ memory reduction at 3 bits per element. The `turboquant/` module evaluates methods of this class on a MTEB retrieval task, measuring the compression versus retrieval accuracy trade-off in the fitness-domain knowledge base.

Chapter 4

METHODOLOGY

This chapter describes the design and implementation of the RAG evaluation pipeline developed in this project. Section 4.1 provides an end-to-end overview. The subsequent sections detail each component: document chunking strategies (Section 4.2), concept normalisation for query enhancement (Section 4.3), cross-encoder reranking (Section 4.4), vector quantisation for memory-efficient retrieval (Section 4.5), parametric knowledge probing (Section 4.6), synthetic data generation (Section 4.7), the construction of a domain-specific triplet corpus and the fine-tuning of the embedding model on it (Sections 4.8 and 4.9), the manually curated test dataset (Section 4.10), and the full evaluation framework (Section 4.11).

4.1 RAG Pipeline Overview

The system implements a modular Retrieval Augmented Generation (RAG) pipeline that ingests authoritative exercise-science literature, indexes it for semantic search, and conditions a large language model’s responses on retrieved passages. The pipeline proceeds through the following stages.

- (i) **Document Ingestion.** More than ten authoritative fitness textbooks and research publications—totalling approximately 250 MB of PDF content—are loaded and parsed into plain text. Sources include publications from the NSCA, ACSM, and peer-reviewed literature on anatomy, periodisation, and muscle hypertrophy.
- (ii) **Chunking.** Each document is split into discrete passages according to one of three strategies (Section 4.2): recursive character splitting, semantic chunking, or parent-child hierarchical chunking.

- (iii) **Embedding.** Each chunk is encoded by a high-dimensional text embedding model (`text-embedding-3-large`, 3,072 dimensions) into a dense vector representation.
- (iv) **Indexing.** All chunk embeddings are stored in an in-memory vector store, enabling sub-second approximate nearest-neighbour search at query time.
- (v) **Query Enhancement.** Before retrieval, the user’s query passes through a concept normalisation module that expands domain abbreviations, decomposes compound queries, and maps lay terminology to canonical technical forms (Section 4.3).
- (vi) **Retrieval.** The normalised query is embedded and the top- K most similar chunks are retrieved by cosine similarity.
- (vii) **Optional Reranking.** A cross-encoder reranker re-scores the top- K candidates and selects the most relevant subset for generation (Section 4.4).
- (viii) **Generation.** The retrieved passages are injected into the prompt context of GPT-4o-mini, which generates a response conditioned exclusively on the retrieved content.
- (ix) **Evaluation.** Outputs are assessed using LLM-as-judge evaluators and information retrieval metrics (Section 4.11).

4.2 Chunking Strategies

Document chunking determines the granularity at which the knowledge base is indexed and retrieved. Three strategies of increasing sophistication are evaluated.

4.2.1 Base Chunking (Recursive Character Splitting)

The baseline strategy recursively splits document text at natural character boundaries in a fixed order of preference: paragraph breaks, newlines, and finally individual characters. The resulting chunks have a maximum window of 1,500 characters with a 150-character overlap between consecutive chunks, ensuring that sentences spanning chunk boundaries are not silently truncated. An alternative configuration with a 500-character window and

50-character overlap is also evaluated to assess the effect of chunk granularity on retrieval quality. This approach requires no embedding calls during indexing and serves as a computationally inexpensive baseline against which semantic methods are compared.

4.2.2 Semantic Chunking

Rather than splitting at fixed character counts, the semantic chunking strategy respects topical coherence. Each document is first segmented at the sentence level. Consecutive sentence pairs are then independently embedded, and the cosine distance between adjacent sentence embeddings is computed:

$$d(e_i, e_{i+1}) = 1 - \frac{e_i \cdot e_{i+1}}{\|e_i\| \|e_{i+1}\|} \quad (4.1)$$

where e_i is the embedding of the i -th sentence. A chunk boundary is placed wherever this distance exceeds the 95th percentile of the distance distribution across the entire document—only the 5% most abrupt topic transitions trigger a split. A one-sentence buffer on each side of the boundary smooths noisy transitions. The result is a set of semantically coherent chunks of variable length, each capturing a thematically unified passage.

4.2.3 Parent-Child (Hierarchical) Chunking

The parent-child strategy decouples the granularity at which chunks are searched from the granularity at which they are returned to the generator.

- **Parent chunks** are produced by the semantic chunking method described above—large, topically coherent segments that provide rich context for generation.
- **Child chunks** are produced by recursively splitting each parent chunk into smaller windows of 400 characters with a 50-character overlap.

At index time, only child chunk embeddings are stored in the vector store. At query time, child chunks are searched for precision, but when a child is retrieved the system returns its corresponding parent chunk to the generator, providing substantially richer context than the narrow child passage alone would afford. To bound memory usage, semantic parent chunks are constructed one document at a time before child splitting is applied.

4.3 Concept Normalisation

Users of a fitness assistant naturally employ informal or abbreviated language: they may write “abs” rather than *rectus abdominis*, use “1RM” for *one repetition maximum*, or ask about “HIIT and steady-state cardio” in a single compound query. Source documents, by contrast, use precise anatomical and physiological terminology. This lexical gap reduces retrieval quality even when relevant passages are present in the index. The concept normalisation module addresses this gap through a two-phase pipeline: a one-time vocabulary extraction phase and a three-stage per-query normalisation phase.

4.3.1 Vocabulary Extraction

Prior to any retrieval, the full text of all source PDFs is processed by a language model to extract a structured domain vocabulary. Each document is divided into token-bounded chunks at paragraph boundaries (up to 12,000 tokens per chunk), and each chunk is submitted to the language model with a structured extraction prompt. The extracted vocabulary covers six categories: exercise types, physiological concepts, training variables, anatomy, conditions and populations, and equipment. Two mapping tables are additionally extracted:

- An **abbreviation table**, mapping short forms to their full expressions (e.g., “HIIT” → “High Intensity Interval Training”; “RPE” → “Rate of Perceived Exertion”; “1RM” → “One Repetition Maximum”).
- A **lay-to-technical table**, mapping colloquial terms to their canonical equivalents (e.g., “abs” → “rectus abdominis”; “lats” → “latissimus dorsi”; “hammies” → “hamstrings”).

Results across all chunks and all documents are merged with case-insensitive deduplication; abbreviation keys are normalised to uppercase, and the longer expansion is preferred when a conflict arises.

4.3.2 Three-Stage Normalisation Pipeline

At query time, each user query passes through three sequential LLM calls, with the extracted vocabulary injected into the system prompt of each call.

Stage 1 — Abbreviation Expansion. The query is inspected for domain abbreviations present in the vocabulary. Recognised abbreviations are replaced with their full forms. The module deliberately does *not* expand abbreviations absent from the vocabulary, adhering to a conservative,

knowledge-base-grounded design principle that prevents hallucinated expansions.

Stage 2 — Coordination Resolution. If the expanded query contains coordinating conjunctions (“and”, “or”) joining distinct fitness concepts, the query is decomposed into independent sub-queries. For example, a query about HIIT and steady-state cardio yields two sub-queries—one per concept—allowing the retrieval system to fetch the most relevant passage for each concept separately.

Stage 3 — Term Variation Handling. Each sub-query is examined for lay terms or non-canonical variants present in the vocabulary. Identified terms are substituted with their canonical technical equivalents. Only vocabulary-grounded substitutions are applied; unrecognised terms are preserved unchanged.

The pipeline produces a list of one or more normalised queries per original input. A structured transformation log recording every decision—including deliberate decisions not to normalise—is emitted at each stage to support traceability. The per-query overhead is three sequential LLM calls.

4.4 Cross-Encoder Reranking

Following the initial retrieval step, an optional cross-encoder reranking stage re-scores the candidate passages. Initial retrieval uses a bi-encoder architecture: the query and each document are encoded independently and relevance is approximated by a cosine score. While computationally efficient, this approach sacrifices some accuracy because the two representations are computed without mutual attention.

The cross-encoder addresses this by scoring each (query, document) pair jointly: both strings are concatenated and passed through a transformer simultaneously, enabling full cross-attention between query and document tokens and producing a more discriminative relevance score. The reranker is applied to the top- K candidates returned by the bi-encoder (default $K = 10$), reordering them and returning the top- k highest-scoring passages (default $k = 5$) to the generator. A state-of-the-art BGE cross-encoder model trained specifically for passage reranking is used for this stage. The two-stage design balances recall (fast bi-encoder) with precision (discriminative cross-encoder), and the reranking stage can be toggled on or off to isolate its contribution during evaluation.

4.5 Vector Quantisation: TurboQuant

As the RAG knowledge base grows, storing full-precision (FP32) embedding vectors for all indexed chunks becomes a significant memory burden. With 3,072-dimensional embeddings across thousands of chunks drawn from ten large PDFs, peak RAM consumption during indexing reached approximately 3.7 GB. Vector quantisation compresses stored embeddings by reducing the number of bits per coordinate, trading a small loss in retrieval accuracy for a substantial reduction in memory footprint.

This project implements *TurboQuant*, a two-stage quantisation algorithm achieving near-optimal MSE distortion for random vectors while providing an unbiased inner-product estimator that enables accurate approximate nearest-neighbour search on quantised vectors.

4.5.1 Stage 1: Random Rotation and Lloyd-Max Quantisation

Random rotation. Each embedding vector $x \in \mathbb{R}^d$ is multiplied by a random orthogonal rotation matrix $\Pi \in \mathbb{R}^{d \times d}$, generated via QR decomposition of a Gaussian matrix (Haar distribution):

$$y = \Pi x \tag{4.2}$$

After rotation, each coordinate of y follows a predictable, approximately symmetric distribution. For $d \geq 64$, the coordinate distribution is well approximated by $\mathcal{N}(0, 1/d)$, making the downstream quantisation problem analytically tractable.

Lloyd-Max quantisation. Each rotated coordinate is independently quantised using a Lloyd-Max optimal scalar quantiser for the known coordinate distribution. The Lloyd-Max algorithm finds the set of centroids $\{c_1, \dots, c_{2^b}\}$ and decision boundaries $\{b_0, \dots, b_{2^b}\}$ that minimise the expected per-coordinate MSE distortion:

$$D = \sum_{i=1}^{2^b} \int_{b_{i-1}}^{b_i} (x - c_i)^2 f(x) dx \tag{4.3}$$

where $f(x)$ is the coordinate PDF. The codebook is solved iteratively: boundaries are updated to midpoints between adjacent centroids, and centroids are updated to the conditional expectations $\mathbb{E}[X \mid b_{i-1} \leq X < b_i]$, repeating until convergence. Each quantised coordinate is stored as a b -bit integer index. Dequantisation maps each index back to its centroid and applies the reverse rotation $\hat{x} = \Pi^\top \hat{y}$.

4.5.2 Stage 2: QJL Residual Correction

Stage 1 produces a reconstruction $\hat{x}$ with residual $r = x - \hat{x}$. For inner-product search, this residual introduces a systematic bias into cosine similarity scores. Stage 2 corrects this bias using a Quantised Johnson-Lindenstrauss (QJL) sketch.

A random Gaussian projection matrix $S \in \mathbb{R}^{m \times d}$ with i.i.d. $\mathcal{N}(0, 1)$ entries is applied to the residual, and only the signs of the projections are stored—one bit per projected dimension:

$$z = \text{sign}(Sr) \quad (4.4)$$

At query time, the inner product between a full-precision query vector q and a compressed key vector k is estimated as:

$$\langle q, k \rangle \approx \langle q, \hat{k} \rangle + \|r\| \cdot \frac{\sqrt{\pi/2}}{m} \cdot \langle Sq, \text{sign}(Sr) \rangle \quad (4.5)$$

The first term is the inner product with the MSE reconstruction; the second is the QJL correction, which makes the estimator unbiased. The total compressed representation per vector requires $(b-1) \cdot d$ bits for the Stage 1 MSE indices, m bits for the Stage 2 QJL signs, and a single scalar for the residual norm $\|r\|$.

Benchmarking. TurboQuant is evaluated on the MTEB SciFact retrieval benchmark under asymmetric search conditions: corpus vectors are quantised at 2, 3, and 4 bits per coordinate while query vectors remain in full precision. Evaluation metrics include memory footprint, retrieval latency, nDCG@10, and compression ratio relative to the FP32 baseline. This directly informs the feasibility of scaling the fitness-domain knowledge base to larger corpora without proportional memory growth.

4.6 Knowledge Probing

A central objective of this project is to determine when a language model relies on its own parametric knowledge rather than the retrieved context—a phenomenon termed *parametric knowledge leakage*. The knowledge probing methodology creates controlled experimental conditions under which leakage can be detected and quantified.

4.6.1 Lexical Obfuscation

After retrieval but before prompt construction, domain-specific terms in both the user query and the retrieved documents are replaced by phonetically

plausible nonsense words. For example, *biceps* becomes *kaskush*, *latissimus dorsi* becomes *fomblur*, and *abduction* becomes *glenthorp*. By substituting all recognisable fitness vocabulary with meaningless placeholders, the language model is denied the opportunity to answer from memorised associations; the only possible source of information is the obfuscated retrieved context.

Three substitution maps of increasing intensity are defined:

- **SOFT** (≈ 64 terms): Core muscle names, fundamental movement terms, and common equipment. Establishes a baseline level of obfuscation.
- **MEDIUM** ($\approx 80+$ terms): Extends SOFT with exercise names, grip types, and movement vocabulary.
- **HARD** ($\approx 150+$ terms): Extends MEDIUM with anatomical directional terms (anterior, posterior, lateral), body-part names, training jargon, and positional modifiers—approaching near-complete obfuscation of domain vocabulary.

Substitutions are applied using case-insensitive regular expressions with longer keys matched before shorter ones to prevent partial replacements (e.g., ensuring *biceps brachii* is replaced before *biceps*). Retrieved documents are deep-copied before substitution, preserving originals for any subsequent reverse mapping.

4.6.2 Probing Experiment

For each test question, the original unobfuscated terms are used to retrieve relevant documents from the vector store. Substitutions are then applied to both the query and the retrieved documents. The model is presented with the obfuscated query together with the obfuscated context and asked to produce three guesses for the real meaning of each nonsense word appearing in the query.

A separate LLM judge compares the guesses against the original question. If any guess correctly identifies the original term or a close synonym, *parametric knowledge leakage is detected*: the model reconstructed the answer from training-time associations despite the obfuscation. An obfuscation-blocked metric (score = 1 when leakage is prevented, = 0 when detected) is recorded via the LangSmith experiment tracking platform. Each example is evaluated over five independent repetitions to account for stochastic variation.

4.7 Synthetic Data Generation

To supplement the manually curated test dataset, a synthetic question set is generated from the source PDFs using the RAGAS framework. A knowledge graph is constructed from two source documents—a fitness textbook and a research paper on exercise progression—by applying three sequential graph transforms: (i) *keyphrase extraction*, in which an LLM identifies salient domain concepts from each document chunk; (ii) *embedding extraction*, in which each graph node is assigned a dense vector representation; and (iii) *cosine-similarity edge construction*, in which undirected edges are added between nodes whose embedding similarity exceeds a threshold of 0.85. The resulting graph captures semantic relationships between exercise-science concepts across both source texts.

Test questions are generated from the knowledge graph using a single-hop query synthesiser operating under a “Fitness Newcomer” persona—a profile that produces accessible, practically oriented questions representative of those a beginner might ask.

4.8 Domain-Specific Triplet Generation

The retrieval components described so far rely on a general-purpose embedding model. A complementary line of investigation asks whether the embedder itself can be *adapted* to the exercise-science domain by fine-tuning it on in-domain data. Fine-tuning a bi-encoder with a contrastive objective (Section 4.9) requires a corpus of (*query*, *positive passage*, *hard-negative passage*) triplets. Because no such labelled corpus exists for the fitness textbooks used here, one is synthesised through a four-stage LLM-assisted pipeline. The pipeline is built to run on a single GCP NVIDIA L4 (24 GB) instance, serving open-weight models through the vLLM inference engine: Gemma 2 9B (4-bit quantised) as the generator and lightweight judge, and DeepSeek-R1-0528-Qwen3-8B—a reasoning model—as the expert hard-negative judge.

4.8.1 Stage 1: Agentic Chunking

Each source document is segmented into semantically coherent chunks rather than fixed-length windows. For every chunk, the language model additionally produces a one-to-two sentence summary, and the result is stored as a JSON Lines record `{id, content, summary}`. These chunks form both the corpus over which hard negatives are later mined and the pool of candidate *positive*

passages for question generation.

4.8.2 Stage 2: Positive Pair Generation

For each content-bearing chunk, two questions are generated in *separate* LLM calls—one in a **formal** (textbook) register and one in an **informal** (casual gym-goer) register—so that the two styles do not bleed into one another. The generator first decides whether the chunk carries substantive content; non-substantive chunks (tables of contents, author credentials, copyright notices, reference lists, figure captions, and other metadata) are skipped. Each generated question is then validated by a *judge* LLM enforcing five hard rules, all of which must pass: (i) **answerability**—the chunk fully and definitively answers the question; (ii) **specificity**—the question targets information unique to the chunk rather than generic trivia; (iii) **no attribution**—the question never names institutions, people, studies, or specific books; (iv) **user persona**—the question never references the source itself (“this book”, “the third edition”); and (v) **style match**—the register matches the requested style. On rejection, the failure reason is fed back to the generator for up to two retries per style. The chunk that produced a question is, by construction, the gold *positive* for that question.

4.8.3 Stage 3: Hybrid Search and Hard-Negative Mining

For each generated question, candidate hard-negative passages are retrieved from the *same book* as the source chunk, with the source chunk itself always excluded. Two retrievers operate in parallel: a sparse BM25 retriever over a lowercase, alphanumeric tokenisation, and a dense retriever using BAAI/bge-m3 with L2-normalised embeddings (cached on disk and keyed on model, identifiers, and content so that unchanged files skip re-encoding). The two ranked lists are combined by Reciprocal Rank Fusion with $k = 60$ [2], chosen over weighted-score fusion precisely because BM25 scores are unbounded while cosine similarity is confined to $[-1, 1]$, making any score-level weighting brittle. A pool of 50 candidates is drawn from each list, fused, and the top five surviving candidates are emitted per query as the inputs to the expert judge.

4.8.4 Stage 4: Expert Judge Validation

Each (query, candidate) pair is labelled by the DeepSeek-R1 reasoning judge as **positive**, **hard_negative**, or **irrelevant**. The output schema is de-

liberately *reason-first*—the JSON `reason` field precedes the `label` field—so that the model commits its chain-of-thought justification before its verdict, which structured-output decoding would otherwise force it to emit first. A **positive** candidate contains a complete, self-sufficient answer to the query; a **hard_negative** shares meaningful lexical or topical overlap with the query but answers a *different, adjacent* question (the same exercise or nutrient but a different goal, scope, or population—“same vocabulary, wrong answer”); an **irrelevant** candidate shares neither. The prompt requires every verdict to quote the specific passage being judged, so that contradictions are caught at write time, and defaults a malformed verdict to **irrelevant** to keep noise out of the training set. The judge was calibrated on fifteen hand-labelled cases at 13/15 (87%) overall accuracy with a perfect 6/6 on the **positive** bucket—the critical bucket, since a false hard negative would teach the model to push apart a query and a passage that genuinely answers it. The labelled records are exploded into (*query, positive, hard_negative*) rows ready for contrastive training.

4.9 Retriever Fine-Tuning

The triplet corpus of Section 4.8 is used to fine-tune the open-weight BAAI/bge-m3 embedding model (and, in the epoch sweep of Section 8.4, three smaller open-weight encoders) with a contrastive objective, after which the fine-tuned encoder is compared against its off-the-shelf baseline on a held-out split. Open-weight encoders are used for this track because fine-tuning requires access to model weights, which the hosted `text-embedding-3-large` model used elsewhere in the pipeline does not expose.

4.9.1 Hard-Negative Filtering

The agentic chunker’s overlapping windows and the judge’s liberal use of **hard_negative** together admit *false negatives*: a mined hard negative that is in fact a paraphrase of the query’s true answer. Training on such a pair would push the query away from a passage that answers it—the failure mode that dragged an early unfiltered run below baseline. A filtering pass therefore removes any hard negative that, when compared against *any* of the query’s positives, clears either a **lexical** threshold (Jaccard overlap ≥ 0.7 over lowercased, alphanumeric, length- > 2 token sets) or a **semantic** threshold (bge-m3 cosine similarity ≥ 0.9). Positives are never modified, so the evaluation ground truth is unaffected.

4.9.2 Train/Test Split

The data is split 80/20 by `source_chunk_id` with a fixed seed. Because every question is generated from exactly one source chunk, partitioning the unique chunk identifiers is equivalent to partitioning the queries, and the splitter explicitly asserts that no query appears on both sides. The corpus itself remains shared at retrieval time, as is standard for retrieval evaluation. For each test query a multi-relevant ground-truth set is formed as the union of its source chunk and any judge-confirmed positives.

4.9.3 Contrastive Training

Training uses Multiple Negatives Ranking Loss (MNRL): for each anchor query, its paired positive is the target and all other passages in the batch act as negatives. A subtle but important invariant is that no query may appear twice in the same batch—otherwise one row’s positive would enter another row’s negative pool as a spurious negative for the same query. All batches are therefore routed through a `NO_DUPLICATES` sampler that defers a row to a later batch if any of its anchor, positive, or negative texts already appears in the current batch. On the GCP L4, the *cached* variant of MNRL (backed by GradCache) decouples the in-batch negative pool from VRAM, allowing a large effective batch—and hence a large negative pool, the principal driver of contrastive signal—to fit in memory; on Apple-silicon (MPS) smoke runs the implementation falls back to plain MNRL automatically. Default hyperparameters are three epochs, batch size 64, learning rate 2×10^{-5} , a warm-up ratio of 0.1, and a maximum sequence length of 512 tokens, with per-epoch snapshots saved so that each epoch is an independent evaluation point.

4.9.4 Evaluation Protocol

The fine-tuned and baseline encoders are compared on the held-out test queries against a shared corpus. To make the comparison fair and in-domain when the training data covers only some books, the corpus is scoped to the book(s) the training data was drawn from. For each query the encoder embeds the corpus once, retrieves the top ten passages by cosine similarity, and records the rank at which any ground-truth passage appears. Reported metrics are **Recall@1**, **Recall@5**, **Recall@10**, and **NDCG@10**, treated as multi-relevant, macro-averaged across queries with percentile-bootstrap 95% confidence intervals (1,000 resamples), and additionally broken down by query style (formal vs. informal).

4.10 Test Dataset

The primary evaluation dataset consists of 36 curated single-hop questions drawn from nine authoritative exercise-science resources: *Bodybuilding Anatomy*, *Exercise Technique Manual*, *Essentials of Strength Training and Conditioning* (NSCA), *Science and Development of Muscle Hypertrophy*, *Periodization: Theory and Methodology of Training*, *Anatomy of Movement*, publications from the ACSM and NSCA, and selected research papers. Each entry includes the question, a ground-truth reference answer, the source resource and page number, a verbatim reference text excerpt, and the question type. The distribution across sources is weighted towards periodisation ($\approx 33\%$), bodybuilding anatomy ($\approx 14\%$), and muscle hypertrophy ($\approx 14\%$), with the remainder spread across the remaining sources.

4.11 Evaluation Framework

4.11.1 LLM-as-Judge Evaluators

Four evaluators assess end-to-end RAG quality using GPT-4o-mini as the judge. Each evaluator enforces a structured output schema consisting of a boolean verdict paired with a natural-language explanation, reducing ambiguity and improving scoring consistency across runs.

- (i) **Correctness.** Given the question, the ground-truth reference answer, and the RAG-generated answer, the judge grades whether the generated answer is factually consistent with the reference. Additional information is permissible provided it introduces no conflicting claims. A step-by-step reasoning prompt is used to avoid premature conclusions.
- (ii) **Relevance.** Given the question and the generated answer, the judge assesses whether the response is concise and directly addresses the user’s query. A factually accurate but off-topic response receives a failing grade.
- (iii) **Groundedness.** Given the retrieved documents and the generated answer, the judge verifies that every claim in the answer is supported by the retrieved context. Any information introduced beyond what the retrieved passages contain—regardless of its factual accuracy—is treated as hallucination. This evaluator most directly measures the system’s core objective of non-parametric grounding.

- (iv) **Retrieval Relevance.** Given the retrieved documents and the question, the judge evaluates whether the retrieved passages collectively contain the specific information needed to answer all parts of the question. Unlike simple topical overlap, this evaluator requires that a reader could derive the complete answer from the retrieved context alone. Few-shot examples are embedded in the prompt to distinguish passages that are merely topically related from those that are directly responsive. This is the most stringent of the four evaluators.

The RAGAS framework provides a complementary set of metrics—answer relevancy, faithfulness, and context relevance—as an alternative evaluation path that enables lightweight iteration without requiring a LangSmith project.

4.11.2 Custom Retrieval Benchmark

To evaluate retrieval quality independently of generation, a custom benchmark is constructed with predefined, graded query-document relevance judgements.

Dataset construction. One hundred distinct fitness topics are generated in batches of ten, with each batch conditioned on all previously generated topics to enforce orthogonality. Topics span exercise technique, training programming, nutrition, recovery, physiology, equipment usage, and injury prevention. For each topic, four types of document are synthesised:

- **One gold document** (relevance score = 2): a 150–300 word passage that fully and directly answers the query, written in canonical scientific terminology using the HyDE approach.
- **Two hard-negative distractor documents** (relevance score = 0): passages sharing the same title, domain vocabulary, and exercise names as the gold document but addressing a different sub-question within the topic. They are constructed to have high lexical similarity to the gold document in order to challenge embedding-based retrieval.
- **One partly-relevant document** (relevance score = 1): a passage providing useful background information for the query but lacking the specific details needed for a complete answer.

The total corpus consists of approximately 400 documents with graded relevance judgements.

Metrics. Retrieval quality is measured using Normalised Discounted Cumulative Gain (nDCG), which rewards placing the most relevant documents

at the top of the ranked list. Let rel_i denote the graded relevance score (0, 1, or 2) of the document at position i :

$$\text{DCG}@k = \sum_{i=1}^k \frac{\text{rel}_i}{\log_2(i+2)} \quad (4.6)$$

$$\text{nDCG}@k = \frac{\text{DCG}@k}{\text{IDCG}@k} \quad (4.7)$$

where $\text{IDCG}@k$ is the DCG achieved by the ideal (best possible) ranking. Additional metrics include $\text{Precision}@k$ (fraction of top- k results that are relevant), $\text{Recall}@k$ (fraction of all relevant documents retrieved in the top- k), and Mean Reciprocal Rank (MRR, the reciprocal of the first relevant document’s rank). Evaluation is conducted with and without cross-encoder reranking to isolate its contribution to ranking quality.

4.11.3 MTEB Fitness Benchmark

To measure the impact of concept normalisation on retrieval quality, a custom MTEB-compatible retrieval benchmark is constructed with a deliberate vocabulary gap between queries and corpus documents.

Dataset construction. A domain vocabulary of approximately 50 lay-term-to-canonical mappings and 50 abbreviation-to-full-form mappings is generated. One hundred orthogonal fitness topics and queries are then produced with the constraint that each query contains at least one lay term or abbreviation from the vocabulary—deliberately simulating informal user input (e.g., “why am I so wrecked after HIIT?”). Corresponding gold documents are generated using canonical terminology without lay terms or abbreviations (e.g., writing “High Intensity Interval Training” in full). Three hard-negative distractor documents per query are generated using the same approach as the custom retrieval benchmark. The dataset is exported in MTEB-compatible format.

Key design principle. The deliberate asymmetry between query vocabulary (informal) and corpus vocabulary (canonical) creates a controlled experimental condition: without normalisation, lexical divergence pulls query and document embeddings apart even when both describe the same concept. Concept normalisation, applied at query encoding time, should bridge this gap.

Evaluation. Two embedders are evaluated on the same retrieval task using cosine similarity search:

- A **normalisation-enhanced embedder** applies abbreviation expansion followed by lay-term substitution to each query before encoding. Corpus documents are encoded without transformation, as they already use canonical terminology.
- A **baseline embedder** encodes queries with no transformation, using the same underlying model.

Both embedders use `text-embedding-3-large`. The primary comparison metrics are nDCG@10, nDCG@5, Recall@10, and Precision@10. A positive delta in nDCG@10 for the normalisation-enhanced embedder confirms that concept normalisation successfully bridges the vocabulary gap between informal user queries and the technical knowledge base.

Chapter 5

REQUIREMENTS SPECIFICATION

5.1 Use Cases

The system is designed for two distinct categories of users: an end user (a fitness enthusiast seeking personalised training guidance) and a researcher (a practitioner evaluating or extending the system’s retrieval and generation capabilities). Figure 5.1 presents the complete use case model.

End User use cases:

- **UC1 — Ask a fitness question:** The user submits a natural-language fitness or training question. The system retrieves the most relevant document chunks from the knowledge hub and generates a grounded answer conditioned solely on those retrieved passages. Multi-turn conversation history is maintained across messages, enabling coherent dialogue within a session.
- **UC2 — Request a workout program:** The user describes a desired workout in natural language. The system extracts a structured intent representation (target muscle group, available equipment, experience level, and desired number of exercises), queries the exercise database, and produces an ordered workout plan whose sequencing and volume recommendations are grounded in the retrieved training rules.

Conversation history maintenance is an inherent part of both chat use cases; it extends UC1 and UC2 rather than constituting a separate actor-visible use case.

Researcher use cases:

- **UC3 — Evaluate chunking strategy:** The researcher selects one of three chunking strategies—fixed-size recursive splitting, embedding-based semantic splitting, or hierarchical parent-child splitting—and runs the evaluation pipeline to measure retrieval and generation quality on a standardised test dataset.
- **UC4 — Evaluate reranker:** The researcher enables or disables the cross-encoder reranking stage and compares retrieval quality metrics across configurations.
- **UC5 — Evaluate concept normalisation:** The researcher measures the impact of multi-stage query normalisation on retrieval performance using a custom fitness retrieval benchmark.
- **UC6 — Create dataset with orthogonal topics:** The researcher generates a synthetic retrieval benchmark with graded relevance judgements, comprising gold documents, hard-negative distractors, and partly-relevant documents.
- **UC7 — Run MTEB retrieval task:** The researcher executes the custom MTEB-compatible retrieval task to compare a normalisation-enhanced embedder against a baseline embedder on standard IR metrics.
- **UC8 — Test knowledge leakage with lexical obfuscation:** The researcher runs the parametric knowledge probing experiment at a chosen obfuscation level (soft, medium, or hard) to quantify the degree to which the model relies on retrieved context versus memorised training knowledge.

5.2 Functional Requirements

- **FR1:** The system shall answer fitness questions using only information retrieved from the knowledge hub; responses shall not draw on unconstrained parametric knowledge.
- **FR2:** The system shall parse a natural-language workout request into a structured intent representation containing target muscle group, experience level, available equipment, and desired number of exercises.
- **FR3:** The system shall filter exercises from a structured exercise database based on the extracted intent and return a ranked, rule-grounded workout plan.

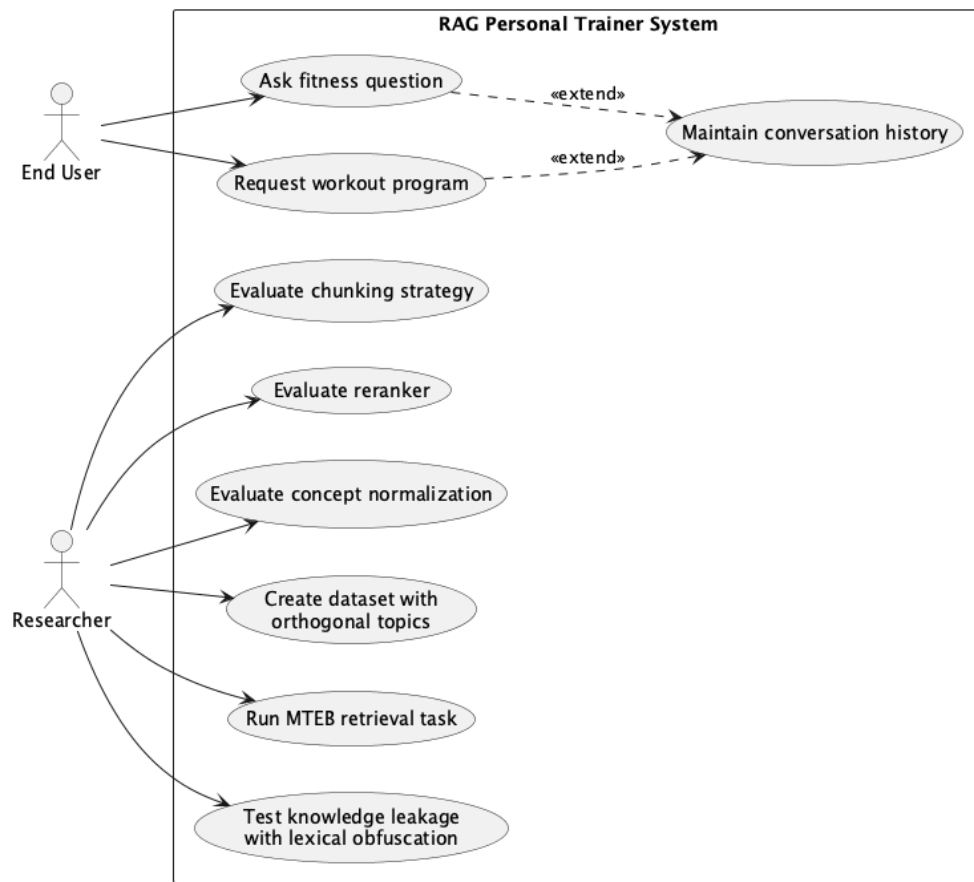


Figure 5.1: Use case diagram for the RAG personal trainer system.

- **FR4:** The system shall support three interchangeable chunking strategies: fixed-size recursive splitting, embedding-based semantic splitting, and hierarchical parent-child splitting.
- **FR5:** The system shall evaluate RAG outputs on four dimensions: correctness, relevance, groundedness, and retrieval relevance.
- **FR6:** The system shall detect parametric knowledge leakage at three lexical obfuscation levels: soft (muscle-name substitution), medium (exercise and equipment substitution), and hard (full obfuscation).
- **FR7:** The system shall optionally apply a cross-encoder reranker to re-score retrieved documents before generation.
- **FR8:** The system shall normalise user queries through abbreviation expansion, coordination resolution, and anatomical term-variation handling prior to embedding.
- **FR9:** The system shall support memory-efficient embedding storage via a two-stage vector quantisation pipeline (random rotation followed by scalar quantisation with residual correction).
- **FR10:** The system shall generate synthetic retrieval benchmarks with graded relevance judgements using a knowledge-graph-based data generation pipeline.
- **FR11:** The system shall synthesise a domain-specific (*query, positive, hard-negative*) triplet corpus—via dual-style question generation, hybrid-search hard-negative mining, and expert LLM judging—and fine-tune an open-weight embedding model on it, evaluating the fine-tuned encoder against its off-the-shelf baseline on a held-out split.

5.3 Non-Functional Requirements

- **NFR1 — Latency:** Chat response latency shall remain below 10 seconds under normal operating conditions (observed mean: approximately 3.3 seconds).
- **NFR2 — Memory efficiency:** Peak RAM consumption during document indexing shall remain within practical bounds for a development workstation (observed peak: approximately 3.7 GB).

- **NFR3 — Cost observability:** Embedding API call costs shall be tracked and reported per evaluation run to support budget management.
- **NFR4 — Reproducibility:** All evaluation runs shall be fully traced and logged to an experiment tracking service, enabling exact reproduction of results.

Chapter 6

DESIGN

6.1 Information Structure

The system does not employ a traditional relational database and therefore a classical entity-relationship (ER) diagram is not applicable. Instead, data is held in the following in-memory and file-based structures:

- **Vector store:** An in-memory vector store holding embedded document chunks. Each chunk is represented as a 3,072-dimensional dense vector produced by a large-scale text embedding model. Similarity search over this store is the primary retrieval mechanism.
- **Exercise database:** A structured tabular dataset containing exercises annotated with attributes including exercise name, difficulty level, target muscle group, available equipment, prime mover, secondary muscles engaged, movement mechanics, and force type. This database is queried programmatically after structured intent extraction.
- **Test dataset:** A curated JSON dataset of 36 question–answer pairs, each accompanied by a ground-truth answer, the reference source document, page number, and question type. This dataset serves as the fixed evaluation benchmark.
- **Knowledge graph:** A graph structure used during synthetic data generation in which nodes represent document chunks annotated with keyphrases and embeddings, and directed edges connect chunks whose embedding cosine similarity exceeds a defined threshold. The graph enables multi-hop question generation over related passages.

6.2 Information Flow

6.2.1 Chat Request Flow

When an end user submits a message, the system executes the following sequence (illustrated in Figure 6.1):

- (i) The user composes a natural-language message in the web interface and submits it.
- (ii) The frontend transmits the message together with the accumulated conversation history to the backend REST API.
- (iii) A retrieval middleware intercepts the request and performs a similarity search against the in-memory vector store, retrieving the top- K ($K = 2$) most relevant document chunks.
- (iv) The retrieved chunks are injected into an augmented prompt alongside the conversation history and a system-level instruction that confines the model to retrieved context.
- (v) The LLM agent generates a response. If the request is identified as a workout program request, the agent additionally invokes a structured intent extraction step to obtain a machine-readable intent representation, queries the exercise database for matching exercises, and orders them according to the training rules retrieved from the knowledge hub.
- (vi) The generated response is returned to the frontend and displayed to the user.

6.2.2 RAG Evaluation Pipeline Flow

The evaluation pipeline proceeds as follows (illustrated in Figure 6.2):

- (i) The authoritative fitness PDFs are loaded from the knowledge hub.
- (ii) Documents are split into chunks according to the selected chunking strategy (fixed-size recursive, semantic, or parent-child).
- (iii) Each chunk is embedded using the configured embedder (baseline or normalisation-enhanced) and indexed in the vector store.
- (iv) For each question in the test dataset, the system retrieves the top- K chunks, optionally applies cross-encoder reranking, and passes the retrieved context to the LLM to generate a response.

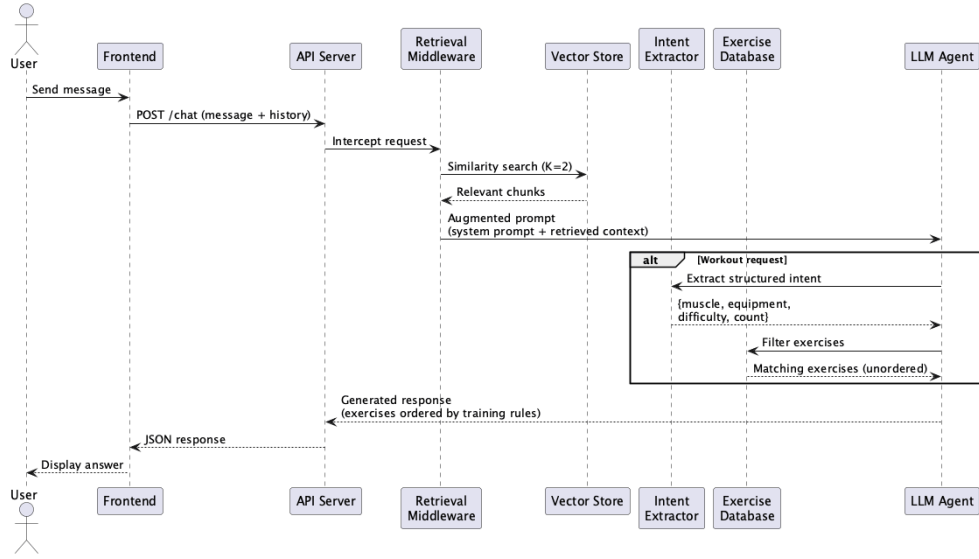


Figure 6.1: Sequence diagram for a chat request in the personal trainer system.

- (v) LLM-as-judge evaluators score each response on correctness, relevance, groundedness, and retrieval relevance.
- (vi) After all questions are processed, results are aggregated and logged to the experiment tracking service.

6.3 System Design

The system is organised into seven loosely coupled modules:

- **Application module:** A REST API backend that exposes a chat endpoint and a health-check endpoint. An LLM agent within this module orchestrates retrieval, intent extraction, exercise filtering, and response generation. A web-based chat frontend communicates with the backend over HTTP.
- **Evaluation module:** The core evaluation harness that coordinates chunking, retrieval, generation, and multi-metric scoring. It contains the following sub-modules:
 - *Chunking strategies:* Three interchangeable chunking implementations sharing a common interface, allowing strategies to be swapped without changes to the surrounding pipeline.

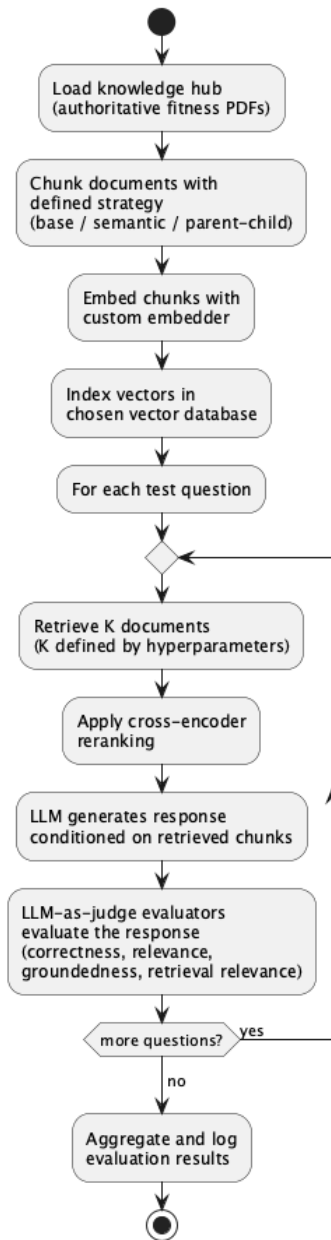


Figure 6.2: Activity diagram for the RAG evaluation pipeline.

- *Cross-encoder reranking*: An optional stage that re-scores retrieved chunks using a cross-encoder model before they are passed to the generator.
 - *Knowledge probing*: A lexical obfuscation sub-module that replaces domain terms at configurable obfuscation levels to probe whether the model relies on retrieved context or memorised knowledge.
 - *Traditional IR metrics*: Computation of nDCG, Precision, Recall, and MRR over retrieved document rankings.
 - *Resource monitor*: Tracks peak RAM consumption and per-call embedding API cost during evaluation runs.
- **Concept normalisation module**: A three-stage query enhancement pipeline consisting of abbreviation expansion, lay-term-to-canonical-term substitution, and coordination resolution. The vocabulary used for substitution is extracted automatically from the source documents.
 - **Vector quantisation module**: An implementation of two-stage embedding compression: a random orthogonal rotation followed by Lloyd-Max scalar quantisation, with a quantisation-aware joint learning (QJL) residual correction step to recover retrieval quality lost during compression.
 - **MTEB benchmark module**: A custom MTEB-compatible retrieval task that encapsulates the fitness corpus and graded relevance judgements, enabling standardised comparison of the normalisation-enhanced embedder against the baseline.
 - **Data generation module**: A synthetic benchmark generation pipeline that constructs a knowledge graph from the source PDFs and uses it to produce single-hop and multi-hop question-answer pairs with graded relevance judgements.
 - **Triplet generation and fine-tuning module**: A four-stage pipeline—agentic chunking, dual-style positive-pair generation, hybrid-search hard-negative mining, and expert LLM judging—that synthesises a domain-specific (*query*, *positive*, *hard-negative*) triplet corpus, together with a contrastive fine-tuning sub-module (false-negative filtering, query-disjoint splitting, cached MNRL training, and held-out evaluation against the off-the-shelf baseline) that adapts an open-weight embedding model to the exercise-science domain.

- **Knowledge hub:** The curated collection of authoritative fitness and exercise science source documents from which all document chunks are derived.

6.4 User Interface Design

The system provides a web-based chat interface through which end users interact with the personal trainer. The interface is built as a single-page application and communicates with the backend via the REST API.

Key interface elements include: a scrollable message history area that visually distinguishes user messages from assistant responses; a text input field and a send button for composing and submitting messages; an animated typing indicator displayed while the backend processes a request; and an inline error message displayed if the backend is unreachable.

Chapter 7

IMPLEMENTATION AND TESTING

7.1 Implementation

The entire system is implemented in Python. The RAG pipeline, including document loading, text splitting, vector store management, retrieval, and agent orchestration, is built with LangChain. All LLM calls and retrieval steps are traced through LangSmith, providing full reproducibility and enabling side-by-side comparison of experiment runs.

OpenAI models serve three roles: generating the synthetic test dataset used for benchmarking, producing dense embeddings via the `text-embedding-3-large` model, and acting as the judge model in the LLM-as-judge evaluation suite. The evaluation framework draws inspiration from RAGAS, an open-source framework for RAG system evaluation; our custom evaluators for correctness, relevance, groundedness, and retrieval relevance are adapted with structured output schemas and few-shot examples tailored to the fitness domain.

Open-source embedding models and a cross-encoder reranker (BGE reranker) are provided by the `sentence-transformers` library. The custom retrieval benchmark for evaluating concept normalization is built using the MTEB (Massive Text Embedding Benchmark) framework, which provides a standardised interface for embedding evaluation tasks. The TurboQuant vector quantisation module—comprising Lloyd-Max codebook construction, random rotation matrices, and QJL projection—is implemented with PyTorch. Text extraction from source PDF textbooks is handled by `pdfplumber`, and Anthropic Claude serves as the language model powering the concept normalisation pipeline (abbreviation expansion, coordination resolution, and term variation handling).

The domain-specific triplet generation and retriever fine-tuning track (Sections 4.8 and 4.9) runs on a GCP NVIDIA L4 (24 GB) instance. Open-weight generator and judge models—`Gemma 2 9B` for question generation and `DeepSeek-R1-0528-Qwen3-8B` for hard-negative judging—are served through the vLLM inference engine with 4-bit (`bitsandbytes`) quantisation; because the L4’s memory cannot hold both at once, the two servers are swapped rather than co-resident. Hard-negative mining fuses a `rank_bm25` sparse retriever with a `BAAI/bge-m3` dense retriever via Reciprocal Rank Fusion. Fine-tuning itself uses the `sentence-transformers SentenceTransformerTrainer` with cached Multiple Negatives Ranking Loss and a `NO_DUPLICATES` batch sampler; the entire pipeline carries a unit-test suite (judge calibration, the hybrid-search ranking logic against a synthetic corpus, and the network-free fine-tuning smoke tests including a direct check of the no-duplicate-batch invariant).

7.2 Testing

7.2.1 Test Dataset

The evaluation dataset consists of 36 curated single-hop questions drawn from nine authoritative sources, including textbooks, peer-reviewed articles, and research papers in the fitness and exercise science domain. Each question is paired with a ground-truth answer and a source reference to support reproducible benchmarking.

7.2.2 Evaluation Experiments

Five experiments were conducted to assess different aspects of retrieval and generation quality:

- (i) **Chunking strategy comparison.** The base chunking strategy was evaluated at two configurations—500-character and 1,500-character windows—across the full test dataset to assess how chunk granularity affects answer quality.
- (ii) **RAGAS evaluation.** Multiple experiment runs measured answer relevancy, faithfulness, and context relevance using the LLM-as-judge evaluators, with results logged to LangSmith for comparison.
- (iii) **Knowledge probing.** Parametric knowledge leakage was tested at three obfuscation levels (soft, medium, and hard), substituting domain

terms with synthetic vocabulary to isolate whether the model relies on retrieved context or memorised training knowledge.

- (iv) **nDCG retrieval benchmark.** A custom benchmark with approximately 400 synthetic documents and graded relevance judgements was used to measure retrieval quality, evaluated with and without cross-encoder reranking.
- (v) **MTEB concept normalisation benchmark.** A normalisation-enhanced embedding pipeline was compared against the baseline on a custom MTEB-compatible fitness retrieval task to quantify the benefit of query preprocessing.
- (vi) **Retriever fine-tuning epoch sweep.** Three open-weight bi-encoders were fine-tuned on a synthetic, domain-specific triplet corpus with cached Multiple Negatives Ranking Loss, and each per-epoch snapshot was evaluated against its off-the-shelf baseline on a held-out, book-scoped split to measure the retrieval gain from domain adaptation and to locate the onset of overfitting.

7.2.3 Observability

All LLM calls were traced through LangSmith, ensuring full reproducibility across experiments. Embedding API costs and peak RAM usage were monitored throughout evaluation runs to assess resource efficiency.

7.3 Deployment

This project is a research prototype and runs locally. No containerisation or cloud deployment infrastructure was introduced, which is appropriate for the research scope. The system is intended to serve as an experimental testbed for evaluating RAG design choices rather than as a production service.

Chapter 8

RESULTS

This chapter presents the experimental results for each component of the RAG pipeline, evaluated independently using the methodologies described in Chapter 4.

8.1 Chunking Strategy Comparison

Figure 8.1 shows the retrieval relevance scores across different chunking configurations. The baseline recursive character splitter was tested at two window sizes (500 and 1,500 characters) and at increasing retrieval depths ($k = 2, 4, 8, 16$). Semantic chunking was evaluated at two percentile breakpoint thresholds (90th and 95th), and the parent-child strategy was evaluated at $k = 16$.

Several trends are visible. First, increasing the retrieval depth k consistently improves retrieval relevance for the base chunker, rising from 0.81 at $k = 2$ to 0.97 at $k = 16$. Second, larger chunk sizes (1,500 vs. 500 characters) provide a modest improvement at the same k , likely because broader context windows are more likely to contain the complete answer passage. Third, semantic chunking at the 95th-percentile threshold achieves a perfect retrieval relevance score of 1.00, outperforming all base configurations. The parent-child strategy reaches 0.97, matching the best base configuration while offering richer context through the hierarchical retrieval mechanism.

8.2 Reranker Evaluation

The impact of cross-encoder reranking was evaluated on a custom retrieval benchmark containing approximately 400 documents with graded relevance

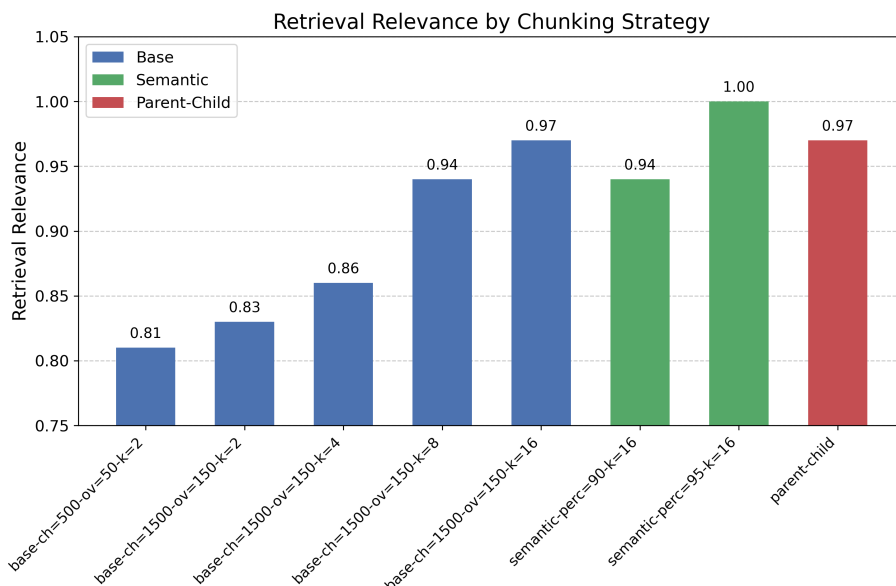


Figure 8.1: Retrieval relevance by chunking strategy and configuration.

judgments. Figure 8.2 shows baseline performance without reranking, while Figure 8.3 compares three reranker model variants.

Without reranking, the embedding-only retriever already achieves strong nDCG scores: 0.93 at $k = 1$, 0.977 at $k = 3$, and 0.978 at $k = 5$. The high baseline reflects the quality of the synthetic benchmark where gold documents are semantically well-separated from distractors. When reranking is applied, the bge-reranker-v2-m3 variant achieves the highest scores across all cut-offs, reaching 0.95 at $k = 1$, 0.9835 at $k = 3$, and 0.9835 at $k = 5$. Notably, the bge-reranker-large variant slightly underperforms the base variant at $k = 1$ (0.885 vs. 0.905), suggesting that larger reranker models do not universally improve top-1 precision. Overall, reranking provides the most benefit at $k = 1$, where the baseline is weakest, while the gains narrow at higher k values where the baseline is already near-ceiling.

8.3 Concept Normalization

The impact of pre-retrieval concept normalization was measured using a custom MTEB-compatible retrieval benchmark, where queries deliberately use lay terms and abbreviations while the corpus uses canonical terminology. Figure 8.4 compares the normalization-enhanced embedder against a baseline that encodes queries without transformation.

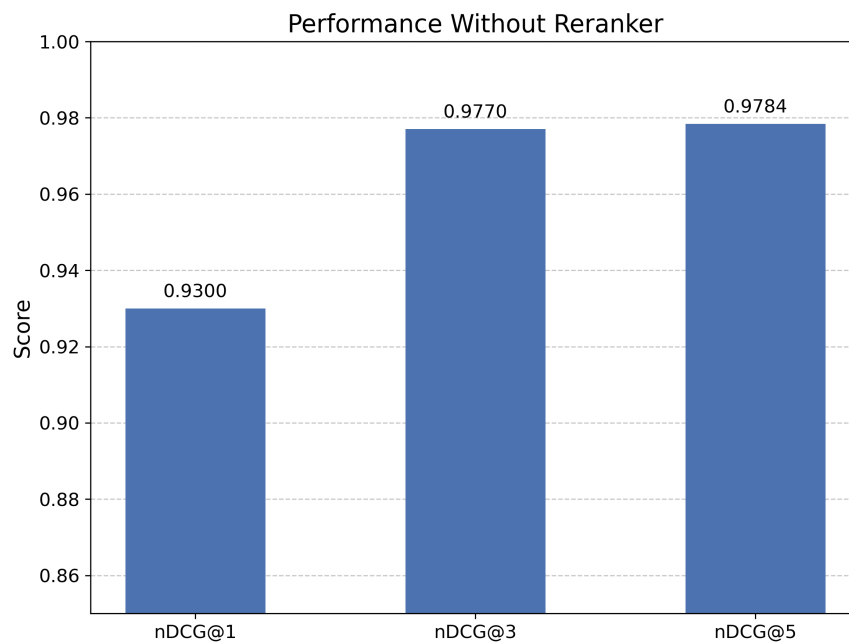


Figure 8.2: Retrieval performance without reranking (nDCG at $k = 1, 3, 5$).

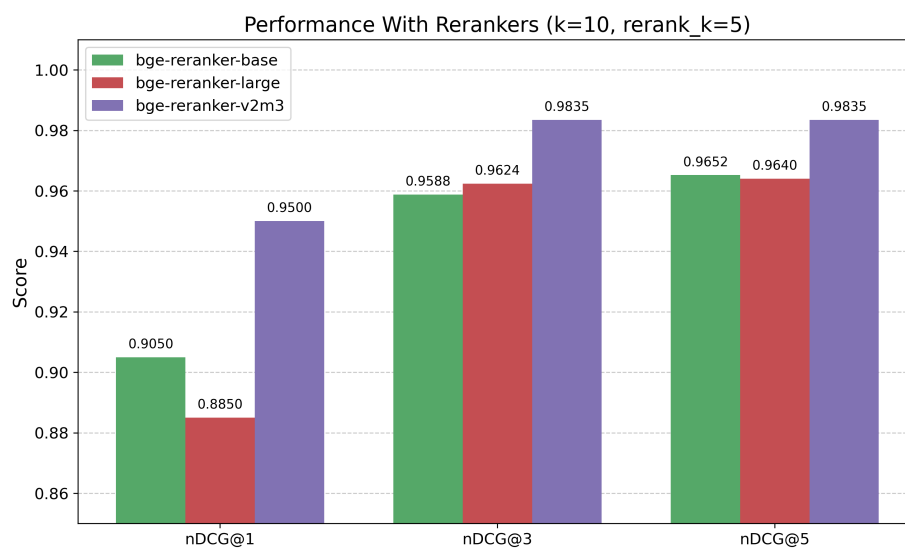


Figure 8.3: Retrieval performance with three cross-encoder reranker variants ($k = 10$, rerank top 5).

The normalization-enhanced embedder achieves a perfect nDCG@10 of 1.0 and recall@10 of 1.0, compared to the baseline’s 0.9963 for both metrics. Precision@10 is identical at 0.10 for both, which is expected given the benchmark design (one gold document per 10 retrieved). While the absolute improvement is small (0.37 percentage points in nDCG), the normalization-enhanced embedder achieves a perfect score, indicating that vocabulary mismatch, though minor for modern large embedding models, is fully eliminated by the normalization step. This suggests that the embedding model already captures most lay-to-canonical relationships, but concept normalization closes the remaining gap.

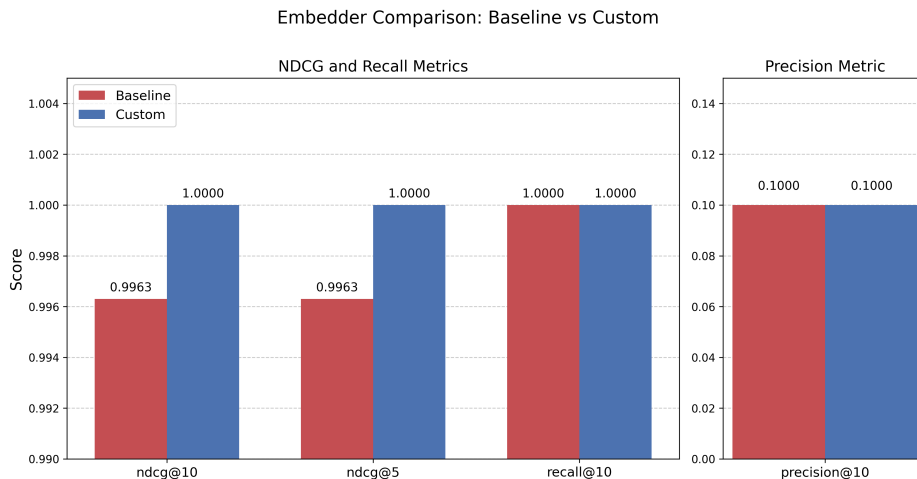


Figure 8.4: Baseline vs. normalization-enhanced embedder on the fitness retrieval benchmark.

8.4 Retriever Fine-Tuning

Whereas concept normalization improves retrieval by transforming the query, retriever fine-tuning improves it by adapting the embedding model itself to the exercise-science domain. Three open-weight bi-encoders of decreasing size were fine-tuned on the domain-specific triplet corpus (Section 4.8) using cached Multiple Negatives Ranking Loss, with a per-epoch snapshot evaluated independently against its off-the-shelf baseline on the held-out, book-scoped test split. The three base models are `bge-small-en-v1.5` (≈ 33.4 M parameters), `all-MiniLM-L6-v2` (≈ 22.7 M), and `bge-micro-v2` (≈ 17.4 M). Table 8.1 reports each model’s baseline against its best fine-tuned epoch.

Three findings stand out. First, **all three models beat their baseline**

Table 8.1: Off-the-shelf baseline vs. best fine-tuned epoch on the book-scoped held-out split (points $\times 100$; best epoch in parentheses).

Model	Baseline R@5	Best R@5 (ep)	Best NDCG@10 (ep)	$\Delta R@5$
bge-small	84.31	86.34 (1)	83.80 (1)	+2.03
MiniLM	79.99	84.29 (1)	82.10 (1)	+4.30
bge-micro	79.39	83.83 (2)	81.70 (2)	+4.44

on every metric at their peak epoch, confirming that fine-tuning on the synthetic in-domain triplets is beneficial and that the hard-negative filtering step (Section 4.9) was sufficient to prevent the false-negative collapse seen in an earlier unfiltered run. Second, **the larger the base model, the higher its absolute score but the smaller its gain**: the strongest off-the-shelf encoder, **bge-small**, improves by only +2.0 Recall@5 because it already operates near the corpus ceiling, whereas the smaller **MiniLM** and **bge-micro**, which start with more headroom, gain roughly +4.3 and +4.4 Recall@5 respectively. Notably, the 17.4M-parameter **bge-micro** closes most of the gap to **bge-small**, reaching an NDCG@10 of 81.7 against the latter’s 83.8 from a substantially weaker starting point, making it the strongest value-for-size choice. Third, **the onset of overfitting is size-dependent**: the two larger models peak at epoch 1 and decline thereafter, while the smallest model tolerates a second epoch before declining. Figure 8.5 shows the per-epoch trajectories for all four metrics with 95% confidence intervals, and Figure 8.6 plots the improvement over baseline directly, making the early peak and subsequent decline visible across every metric and model.

Figure 8.7 relates the training dynamics to the retrieval metric. In each subplot the training loss (left axis) falls steeply and continuously, yet the held-out NDCG@10 (right axis) peaks early and then declines—a textbook signature of overfitting that the training loss alone does not reveal. This divergence is the reason the overfitting decision is taken from the held-out NDCG@10 rather than from any loss curve: a held-out MNRL eval loss, logged for the smallest model, proved near-flat by construction (in-batch contrastive loss contrasts only the few negatives present in each eval batch, whereas NDCG@10 ranks the entire corpus) and was too blunt a signal to act on. Figure 8.8 breaks Recall@5 down by query style; the fine-tuned encoders improve retrieval for both the formal (textbook-register) and informal (casual gym-goer) query styles, confirming that the dual-style question generation of Stage 2 did not bias the encoder toward a single register.

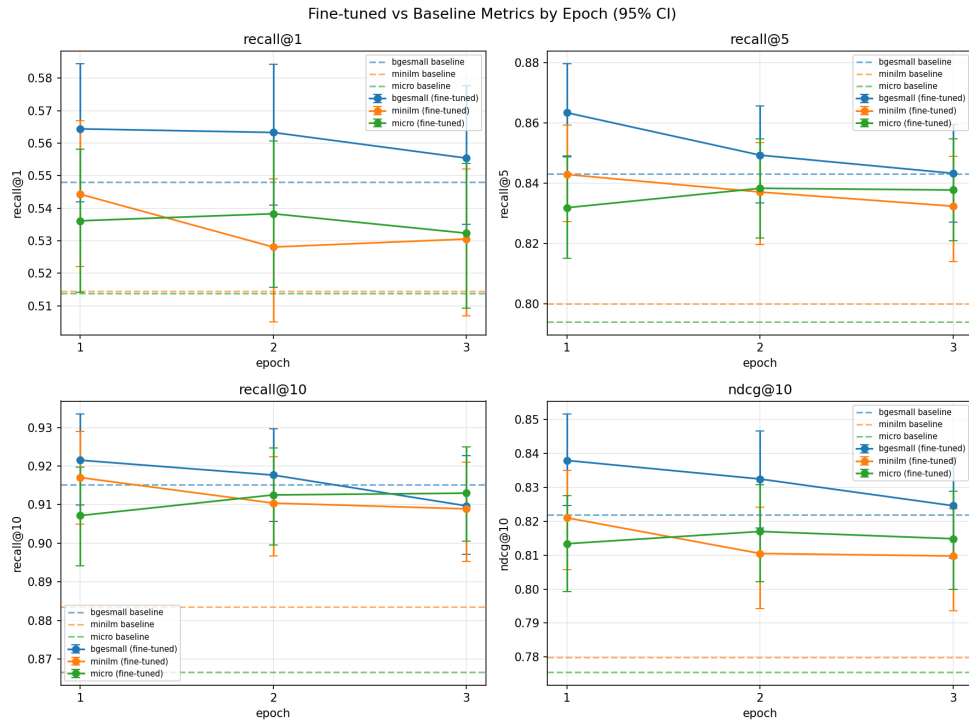


Figure 8.5: Fine-tuned vs. baseline retrieval metrics by epoch, with 95% bootstrap confidence intervals. Solid lines are the fine-tuned models (ordered largest to smallest); dashed horizontal lines are the corresponding off-the-shelf baselines.

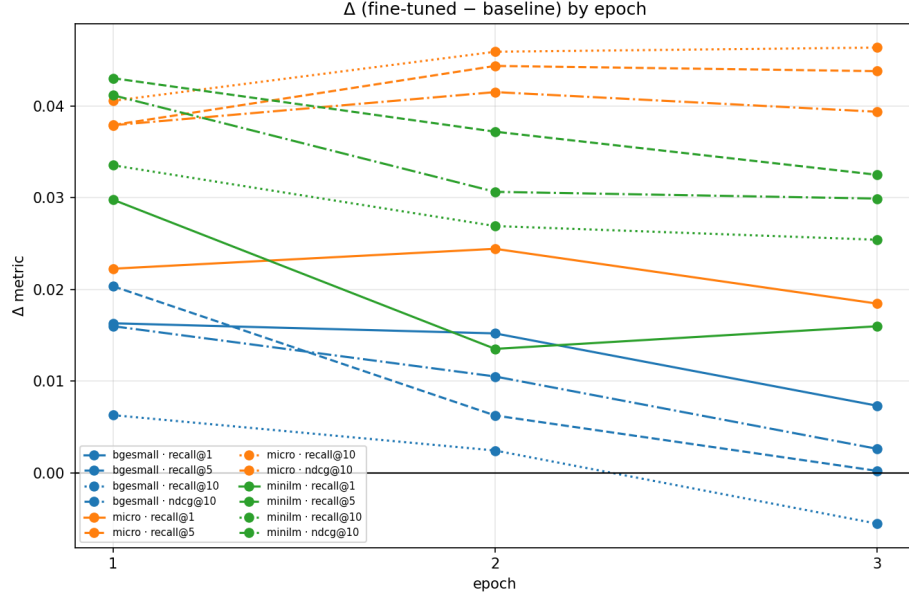


Figure 8.6: Improvement of each fine-tuned model over its baseline ($\Delta = \text{fine-tuned} - \text{baseline}$) by epoch, across all four metrics. The smaller models (orange, green) gain more than the near-ceiling **bge-small** (blue); gains are largest at the early peak epoch and erode with further training.



Figure 8.7: Training loss (left axis, blue) and held-out NDCG@10 (right axis, orange) by epoch, one subplot per model in descending size order. The baseline NDCG@10 is drawn as a dashed reference. Training loss falls monotonically while held-out NDCG@10 peaks early, evidencing overfitting.

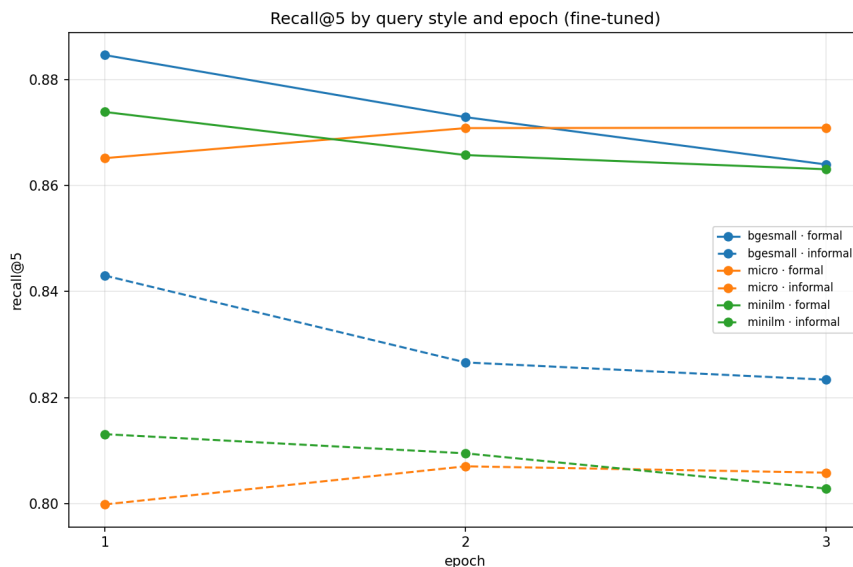


Figure 8.8: Recall@5 of the fine-tuned models by query style (formal vs. informal) and epoch. Both registers benefit from fine-tuning.

8.5 Knowledge Probing via Lexical Obfuscation

Figure 8.9 shows the obfuscation blocked rate across three substitution levels. A higher blocked rate means the model could not recover the original domain terms from their obfuscated forms, indicating successful suppression of parametric knowledge.

At the easy substitution level, only 28% of obfuscated terms were successfully blocked — the model correctly guessed the original terms in 72% of cases, revealing substantial parametric knowledge of fitness concepts. At the medium level, the blocked rate rises sharply to 96%, indicating that extending the substitution map to cover exercises, grips, and movement types effectively prevents the model from leveraging its training knowledge. At the hard level, the blocked rate reaches 100%: with all anatomical directions, body parts, and training-specific jargon replaced, the model is entirely unable to reverse-engineer the original terminology. These results confirm that large language models possess significant domain knowledge in exercise science, and that progressively more aggressive obfuscation is required to isolate the model’s reliance on retrieved context from its parametric memory.

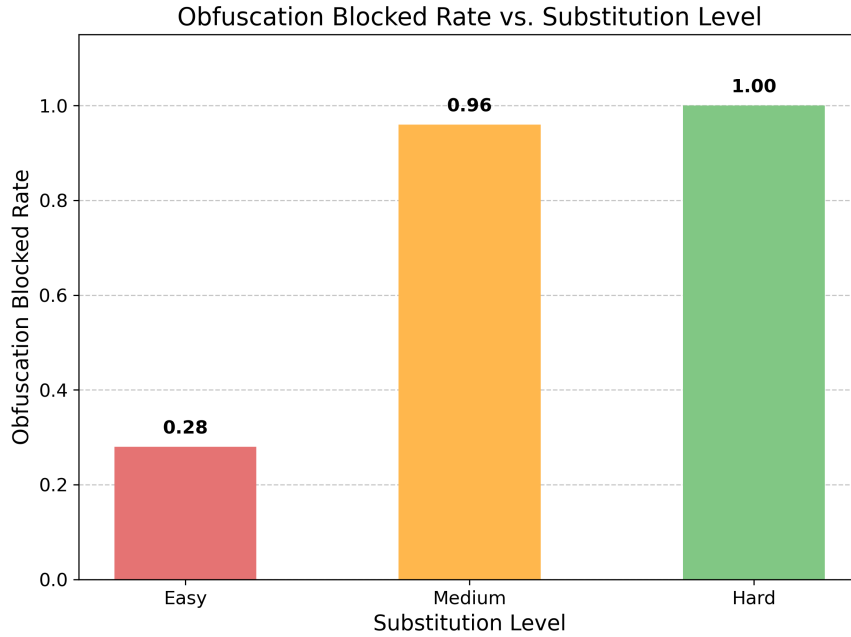


Figure 8.9: Obfuscation blocked rate by substitution level.

8.6 Vector Quantization (TurboQuant)

The TurboQuant vector quantization algorithm was benchmarked on a standard retrieval dataset, measuring the trade-off between compression and retrieval quality. Figures 8.10, 8.11, and 8.12 present the memory footprint, retrieval accuracy, and query speed respectively.

The memory results demonstrate dramatic compression: the full-precision baseline requires 7.59 MB, while 4-bit quantization reduces this to 1.20 MB ($6.3\times$ compression), 3-bit to 0.96 MB ($7.9\times$), and 2-bit to 0.72 MB ($10.5\times$). Retrieval speed improves correspondingly, from 0.289 seconds per query at full precision to 0.064 seconds at 3-bit ($4.5\times$ faster). However, this compression comes at a cost to retrieval accuracy: nDCG@10 drops from 0.5267 at full precision to 0.5129 at 4-bit (2.6% relative loss), 0.4596 at 3-bit (12.7% loss), and 0.3667 at 2-bit (30.4% loss). The 4-bit configuration offers the best trade-off, achieving substantial memory and speed gains with only a minor accuracy degradation. At 2-bit, the accuracy loss becomes significant and may not be acceptable for applications where retrieval quality is critical.

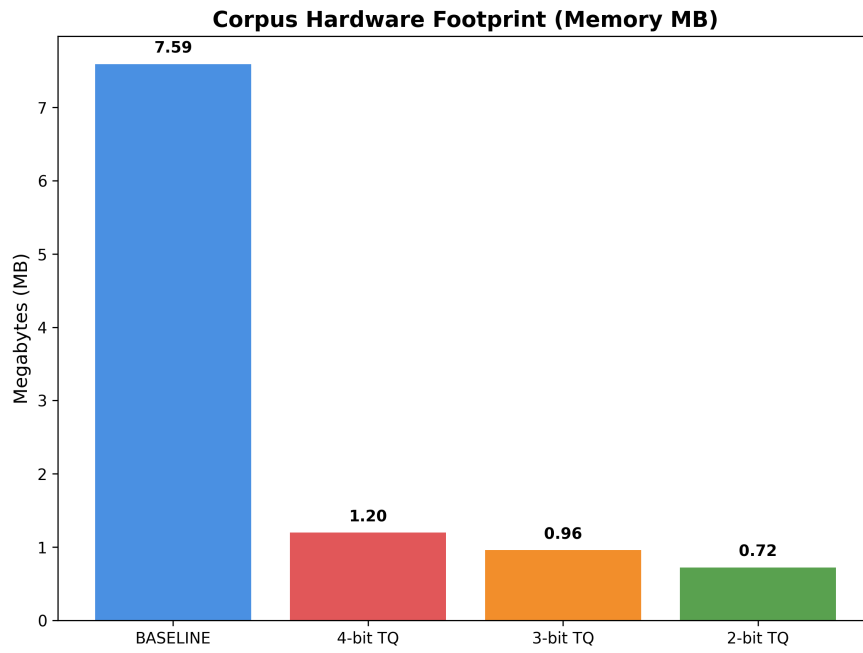


Figure 8.10: Corpus memory footprint at different quantization bit-widths.

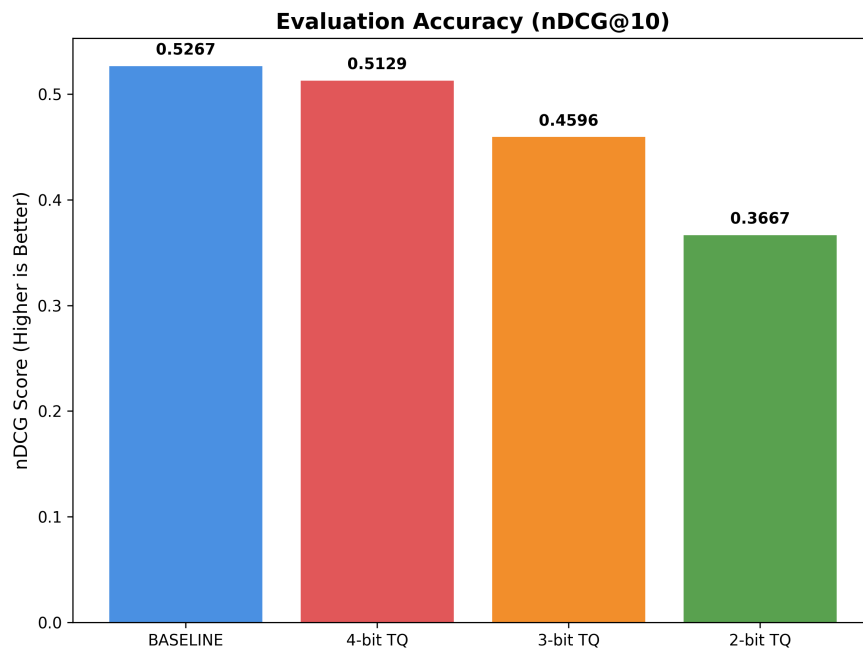


Figure 8.11: Retrieval accuracy (nDCG@10) at different quantization bit-widths.

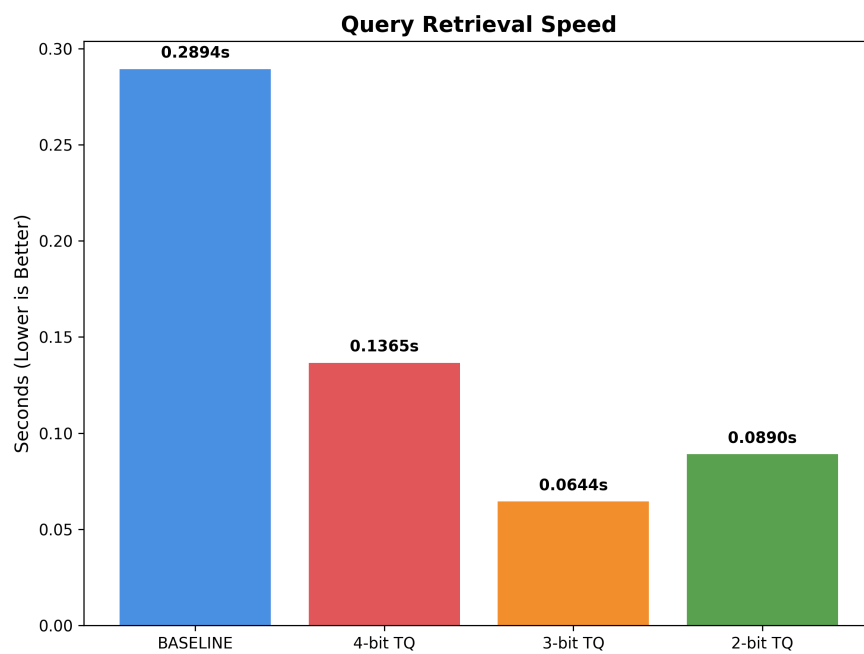


Figure 8.12: Query retrieval speed at different quantization bit-widths.

Chapter 9

CONCLUSION

This project investigated the design, implementation, and evaluation of a Retrieval-Augmented Generation pipeline applied to the exercise science domain. The system was built upon authoritative fitness textbooks and evaluated across multiple dimensions: chunking strategies, cross-encoder reranking, concept normalization, parametric knowledge probing, and vector quantization for memory efficiency.

The system was further extended with a domain-adaptation track: a four-stage LLM-assisted pipeline synthesised a domain-specific triplet corpus from the fitness textbooks—through dual-style question generation, hybrid-search hard-negative mining, and a reasoning-model expert judge—on which open-weight embedding models were fine-tuned with a contrastive objective. This fine-tuning improved retrieval over the off-the-shelf baseline for all three encoders tested (up to +4.4 Recall@5 for the smallest model), with the gain inversely related to model size and the onset of overfitting setting in after a single epoch for the larger encoders. A decisive lesson from this track was that hard-negative *quality* matters more than quantity: an unfiltered corpus, in which mined negatives were sometimes paraphrases of the true answer, drove fine-tuning below baseline, and only after a lexical-and-semantic false-negative filter did fine-tuning yield consistent gains.

The results demonstrate that each component contributes to pipeline quality, but the breadth of the system also highlights a key challenge: a RAG pipeline involves many interacting components — retriever, chunker, embedder, reranker, normalizer, generator, and evaluator — each with its own hyperparameters and design choices. Evaluating these components in isolation, as done in this work, provides clear insights into individual contributions, but does not capture the compounding effects when components are combined. In practice, the optimal chunking strategy may depend on whether reranking is enabled, and the benefit of concept normalization may

vary with the embedding model’s pre-existing vocabulary coverage.

A further challenge lies in the evaluation methodology itself. The retrieval stage and the generation stage require fundamentally different metrics: nDCG, precision, and recall measure how well the retriever ranks relevant documents, while correctness, groundedness, and faithfulness assess whether the generator produces accurate, grounded answers. These two families of metrics can paint different pictures — a retriever may surface relevant documents that the generator then ignores or misinterprets. The knowledge probing results underscore this tension: even when the retriever provides correct context, the generator may rely on its parametric knowledge instead, particularly at lower obfuscation levels.

Looking forward, several directions remain open. The evaluation of combined component configurations (e.g., semantic chunking with reranking and concept normalization simultaneously) would provide a more holistic picture of pipeline quality. The knowledge probing methodology could be extended to a larger question set and additional domains beyond exercise science. The retriever fine-tuning track could be scaled from the smaller open-weight encoders to a full **bge-m3** fine-tune with a larger cached-loss batch on CUDA, and its interaction with the cross-encoder reranker and with concept normalization—each of which also targets retrieval quality—remains to be measured end to end. Finally, the vector quantization results suggest that 4-bit compression offers a practical operating point, but its interaction with reranking and semantic chunking has not yet been explored.

Bibliography

- [1] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics (ACL)*, 2024. Beijing Academy of Artificial Intelligence; University of Science and Technology of China.
- [2] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.
- [3] Kenneth Enevoldsen, Isaac Chung, Imene Kerboua, Márton Kardos, Ashwin Mathur, David Stap, Jay Gala, Wissam Siblini, Dominik Krzemiński, Genta Indra Winata, Saba Sturua, Saiteja Utpala, Niklas Muennighoff, et al. MMTEB: Massive multilingual text embedding benchmark. In *International Conference on Learning Representations (ICLR)*, 2025.
- [4] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, Malta, 2024.
- [5] Insu Han, Praneeth Kacham, Amin Karbasi, Vahab Mirrokni, and Amir Zandieh. PolarQuant: Quantizing KV caches with polar transformation. *arXiv preprint*, 2025. KAIST; Google Research; Yale University.
- [6] Steve Han, Gilberto Titericz Junior, Tom Balough, and Wenfei Zhou. Judge’s verdict: A comprehensive analysis of LLM judge capability

- through human agreement. *arXiv preprint*, 2025. NVIDIA Corporation. Under review as a conference paper at ICLR 2026.
- [7] Ning Kang, Bharat Singh, Zubair Afzal, Erik M. van Mulligen, and Jan A. Kors. Using rule-based natural language processing to improve disease normalization in biomedical text. *Journal of the American Medical Informatics Association*, 20:876–881, 2013. Department of Medical Informatics, Erasmus University Medical Center, Rotterdam, The Netherlands.
 - [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. Facebook AI Research; University College London; New York University.
 - [9] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, 2023. Hugging Face; cohere.ai.
 - [10] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019. Ubiquitous Knowledge Processing Lab, TU Darmstadt.
 - [11] Dongyu Ru, Lin Qiu, Xiangkun Hu, Tianhang Zhang, Peng Shi, Shuaichen Chang, Cheng Jiayang, Cunxiang Wang, Shichao Sun, Huanyu Li, Zizhao Zhang, Binjie Wang, Jiarong Jiang, Tong He, Zhiguo Wang, Pengfei Liu, Yue Zhang, and Zheng Zhang. RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation. *arXiv preprint*, 2024. Amazon AWS AI; Shanghai Jiaotong University; Westlake University.
 - [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. Google Brain; Google Research; University of Toronto.
 - [13] Amir Zandieh, Majid Daliri, and Insu Han. QJL: 1-bit quantized JL transform for KV cache quantization with zero overhead. *arXiv preprint*, 2024. July 2024.